Step 7: Check the *godoc* installation by issuing the following command:

```
$ godoc
usage: godoc package [name …]
…
```

This displays *godoc help*. Here, it's truncated after the first line.

Step 8: Set the required environment variables for Go development, as follows:

```
$ export PATH=$PATH:$GOPATH/bin
$ export GOBIN=$GOPATH/bin
```

Step 9: Add all the export commands executed so far in the *$HOME/.bashrc* file to make them permanent. Now, log out and log in to get all the environment variables updated.

Our first Go program

Let's write our first Go program, a simple implementation of a stack data structure of integers, i.e., a 'last in, first out' list of integers. In this version, I have intentionally avoided all the bells and whistles of the Go language in order to help readers first understand the basics. We will update our program gradually to learn different concepts of the Go language.

Go code is organised in packages. A package can span over multiple *.go* files. We have two files in this example—the stack package in *$GOPATH/src/gostack/stack/stack.go*, and the main package, which contains the *main() function*, the entry point of our program in *$GOPATH/src/gostack/gostack.go*. Let's go through our *stack.go* code first.

```
 $GOPATH/src/gostack/stack/stack.go
// The code in this file belongs to the 'stack' package
package stack
// Import the required packages, here only 'error'
import "fmt"
// C reate a 'Stack' type, which is a slice of integers
type Stack []int
// 'Push' function to push item onto the stack given as
argument
func Push(stack *Stack, item int) {
     *stack=append(*stack,item)
}
// 'Pop' function to pop item from the stack given as
argument
func Pop(stack *Stack,item *int) bool {
     if len(*stack)==0 {  // len() returns the no of elements
in a slice
       fmt.Println("Pop:: The stack is empty")
       return false
     }
     // remove the last element from the slice in item
     *item=(*stack)[len(*stack)-1]
     // creates a new slice by taking the ele from first to
second last
     *stack=(*stack)[:len(*stack)-1]
     return true
}
// 'Show' function to display the content of the stack
func Show(stack Stack) {
     if len(stack)!=0 {
       fmt.Printf("[ ")
       for _,item := range (stack) {
           fmt.Printf("%v ",item)
       }
       fmt.Printf("]  ")
     } else {
       fmt.Println("Show:: The stack is empty")
     }
}
```

The first *go* statement states the package name 'Stack', which our code belongs to. Then we need to import a set of built-in or custom packages that are needed in our program; here, only the *fmt* package, required for formatted I/O, has been imported. Now, we have created a custom type 'Stack' which is equal to a slice of integers. We have written three functions to operate on a stack -- *Push()*, *Pop()* and *Show()*. The *Push()* function takes a Stack pointer and an integer item to push onto the stack. Since a slice, unlike an array, can grow indefinitely, we skipped the overflow check. The *Pop()* function takes a Stack pointer and an integer pointer item, where the popped item is returned. It returns a Boolean value to indicate the success or failure of the operation. The *Show()* function takes a stack of value type. It prints the values present in the stack to the console.

Note: The *Push()* and *Pop()* functions take the address of the stack to be operated on, because they need to modify the original stack. But, the *Show()* function doesn't modify the original stack, so it can operate on a copy of it and it is passed as a value. For custom type of a bigger size, it is efficient to use call-by-address, even for 'read only' use cases to avoid copying of a big chunk of memory.

Now let's see our main package, shown below:

```
$GOPATH/src/gostack/gostack.go
// The code in this file belongs to the 'main' package
package main

// import our 'stack' and other required packages
import (
     "gostack/stack"
     "fmt"
)
```